

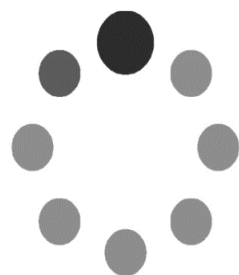


Curso: Desenvolvimento de Software Multiplataforma

Disciplina: Engenharia de Software II

Professor: André Olímpio

Engenharia de Software II

 **LOADING** 

Casos de Testes

- Em projetos de desenvolvimento de software, a garantia da qualidade é um dos aspectos mais críticos.
- Para assegurar que o sistema atenda aos requisitos e funcione corretamente sob diversas condições, é necessário realizar testes sistemáticos.
- Dentro desse processo, os **casos de teste** são ferramentas fundamentais que guiam a verificação da conformidade e do comportamento do software.

Casos de Testes

- **Caso de Teste (Test Case):** é um conjunto estruturado de condições, entradas, procedimentos e resultados esperados, criado com o objetivo de validar uma funcionalidade específica de um sistema de software.
- Cada caso de teste tem como função verificar se uma funcionalidade está funcionando conforme especificado, geralmente com base em requisitos técnicos ou de negócio.

Casos de Testes

- **Um caso de teste é constituído por:**

- O cenário de teste (o que está sendo testado);
- As entradas ou condições iniciais;
- As ações que o testador deve executar;
- O resultado esperado (o comportamento correto do sistema);
- O resultado obtido após a execução do teste.

SOFTWARE TESTING



Fonte: Sisnema.com.br

Conceitos de Testes

- **Teste:** é a atividade prática de verificar se um software atende aos requisitos. Pode envolver a execução de um ou mais casos de teste. É a ação de testar.
- **Caso de Teste (Test Case):** é o documento ou artefato que detalha como o teste será realizado, contendo os passos, entradas e resultados esperados.
- **Plano de Teste (Test Plan):** é um documento de alto nível que define a estratégia geral de testes, incluindo o escopo, objetivos, recursos, cronograma, tipos de testes, critérios de entrada e saída e responsabilidades da equipe.

Conceitos de Testes

- O **plano de teste** orienta como os testes devem ser organizados.
- Os **casos de teste** detalham os cenários específicos.
- A **execução dos testes** aplica esses casos para validar o sistema.



Fonte: Tipteten.com.br

Ciclo de Vida dos Testes

- **Análise de Requisitos:**

- Compreensão dos requisitos funcionais e não funcionais.
- Identificação de pontos críticos que exigem validação.

- **Planejamento de Testes:**

- Criação do plano de teste.
- Definição de escopo, ferramentas, equipe, ambiente de teste e cronograma.

Ciclo de Vida dos Testes

- **Design de Casos de Teste:**

- Elaboração dos casos de teste com base nos requisitos e nos critérios de aceitação.
- Definição de dados de teste e condições iniciais.

- **Configuração do Ambiente de Testes:**

- Preparação dos ambientes (servidores, bancos de dados, aplicações) necessários para executar os testes.

- **Execução dos Testes:**

- Realização prática dos testes conforme os casos definidos.
- Registro dos resultados obtidos (sucesso ou falha).

Ciclo de Vida dos Testes

- **Registro e Reporte de Defeitos:**
 - Notificação de bugs ou falhas encontradas.
 - Comunicação com os desenvolvedores para correção.
- **Reexecução e Testes de Reteste:**
 - Após as correções, os testes são reexecutados para verificar se o defeito foi resolvido.
- **Encerramento e Relatório Final:**
 - Avaliação dos testes executados, métricas de sucesso/falha e lições aprendidas.
 - Documentação do resultado do processo.

Casos de Testes

- Entender o papel dos **casos de teste** é essencial para garantir a qualidade de um sistema.
- Fornecem uma maneira padronizada de verificar funcionalidades específicas e evitam que falhas passem despercebidas durante o desenvolvimento.
- Ao mesmo tempo, **saber diferenciar teste, caso de teste e plano de teste** permite uma organização mais clara da estratégia de qualidade.
- Por fim, **seguir o ciclo de vida dos testes** garante que os esforços de validação sejam consistentes, rastreáveis e eficazes ao longo do processo de desenvolvimento de software.

Ciclo de Vida de um Teste

- **Análise de Requisitos:**

- Compreensão dos requisitos funcionais e não funcionais.
- Identificação de pontos críticos que exigem validação.

- **Planejamento de Testes:**

- Criação do plano de teste.
- Definição de escopo, ferramentas, equipe, ambiente de teste e cronograma.

Ciclo de Vida de um Teste

- **Design de Casos de Teste:**

- Elaboração dos casos de teste com base nos requisitos e nos critérios de aceitação.
- Definição de dados de teste e condições iniciais.

- **Configuração do Ambiente de Testes:**

- Preparação dos ambientes (servidores, bancos de dados, aplicações) necessários para executar os testes.

Ciclo de Vida de um Teste

- **Execução dos Testes:**
 - Realização prática dos testes conforme os casos definidos.
 - Registro dos resultados obtidos (sucesso ou falha).
- **Registro e Reporte de Defeitos:**
 - Notificação de bugs ou falhas encontradas.
 - Comunicação com os desenvolvedores para correção.

Ciclo de Vida de um Teste

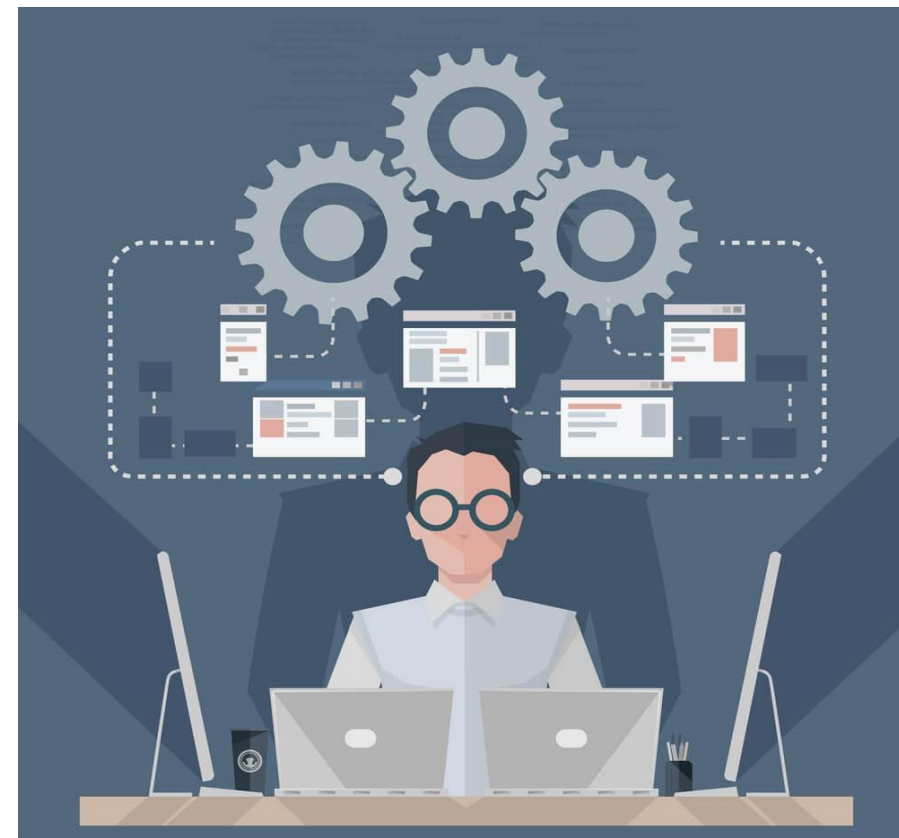
- **Reexecução e Testes de Reteste:**
 - Após as correções, os testes são reexecutados para verificar se o defeito foi resolvido.
- **Encerramento e Relatório Final:**
 - Avaliação dos testes executados, métricas de sucesso/falha e lições aprendidas.
 - Documentação do resultado do processo.

Boas Práticas na Escrita de Casos de Teste

- Os casos de teste são documentos fundamentais no processo de validação de sistemas de software.
- Quando bem escritos, contribuem para a identificação de falhas, a verificação da conformidade com os requisitos e o aumento da confiabilidade do produto final.
- No entanto, a eficácia dos testes depende diretamente da qualidade com que esses casos são elaborados.

Boas Práticas na Escrita de Casos de Teste

- **Clareza e Objetividade:**
 - Um caso de teste bem escrito deve ser claro, direto e livre de ambiguidades.
 - Isso garante que qualquer membro da equipe consiga compreender e executar o teste da mesma forma.



Fonte: CedroTech.com

Boas Práticas na Escrita de Casos de Teste

- **Boas práticas para garantir clareza e objetividade:**
 - Utilize linguagem simples, técnica e precisa.
 - Descreva o objetivo do teste de forma concisa.
 - Evite termos vagos como *“verificar se funciona corretamente”* e prefira *“verificar se o sistema exibe a mensagem de confirmação ao salvar o cadastro”*.
 - Utilize estruturas padronizadas para facilitar a leitura (título, pré-condições, passos, dados de entrada, resultado esperado).

Boas Práticas na Escrita de Casos de Teste

- **Exemplos:**

- **Exemplo ruim:**

- ✓ “Testar o botão de salvar.”

- **Exemplo claro:**

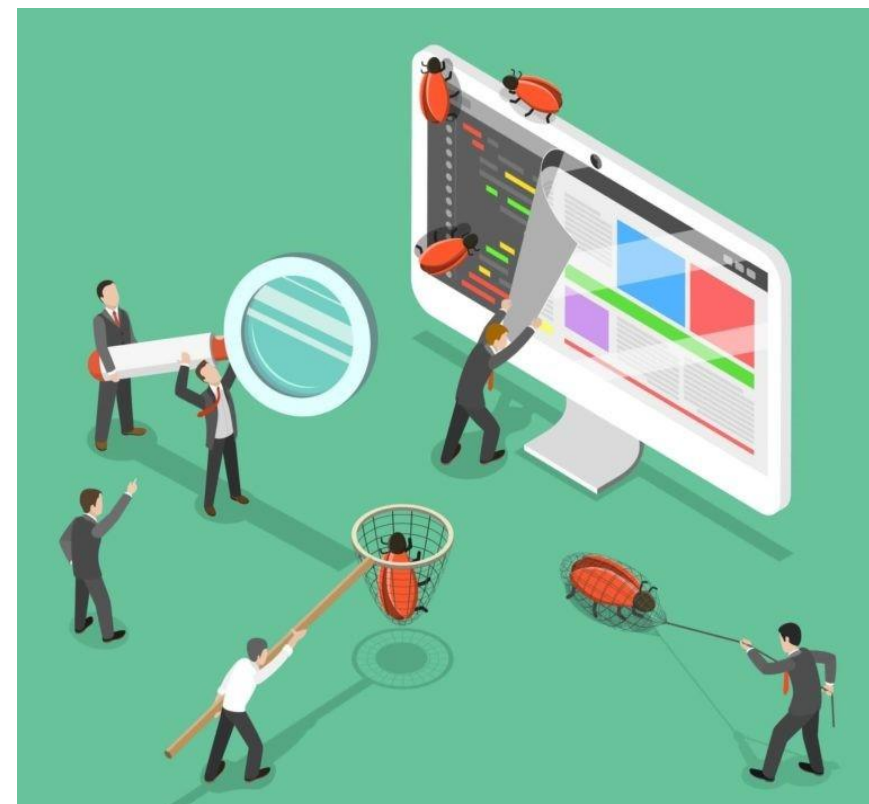
- ✓ “Verificar se o sistema salva os dados do formulário de cadastro ao clicar no botão ‘*Salvar*’ com todos os campos obrigatórios preenchidos corretamente.”



Fonte: Compraco.com.br

Reprodutividade

- Um caso de teste precisa ser reproduzível, ou seja, deve ser possível executá-lo várias vezes obtendo os mesmos resultados esperados, sob as mesmas condições.
- Isso garante consistência nos testes e facilita a identificação de regressões no software.



Fonte: RockApps.com.br

Reprodutividade

- **Como garantir a reprodutibilidade:**
 - Defina pré-condições claras, como usuários logados, permissões ou estados do sistema.
 - Informe dados de entrada específicos (**Ex:** “Nome: João Silva, CPF: 123.456.789-00”).
 - Detalhe cada passo de execução, evitando suposições.
 - Evite dependências externas não controladas (como conexões instáveis ou dados variáveis do ambiente).

Cobertura de Cenários Positivos e Negativos

- Para que os testes sejam eficazes, os casos devem contemplar tanto **cenários positivos** (funcionamento esperado) quanto **cenários negativos** (comportamentos incorretos, erros ou falhas de uso).
- **Cenários positivos:**
 - Validam se o sistema funciona corretamente quando tudo está conforme o esperado.
 - **Exemplo:** salvar cadastro com todos os campos obrigatórios preenchidos corretamente.

Cobertura de Cenários Positivos e Negativos

- **Cenários negativos:**
 - Avaliam como o sistema reage a condições adversas ou inválidas.
 - **Exemplo:** tentar salvar cadastro com um campo obrigatório vazio ou com dado em formato inválido.
- **Por que isso é importante?**
 - Porque muitos erros ocorrem justamente quando os usuários interagem de forma imprevista com o sistema. Antecipar esses cenários ajuda a construir sistemas mais robustos.

Documentação e Versionamento

- A documentação adequada dos casos de teste facilita o gerenciamento, manutenção e reuso ao longo do tempo.
- Além disso, aplicar boas práticas de versionamento permite acompanhar alterações e manter a rastreabilidade com os requisitos do sistema.



Fonte: Flaticon.com

Documentação e Versionamento

- **Boas práticas para documentação e versionamento:**
 - Utilize ferramentas de gestão de testes (como TestLink, Zephyr, Azure Test Plans ou planilhas padronizadas).
 - Inclua identificadores únicos (**Ex:** CT-001, CT-002).
 - Mantenha o histórico de edições, com autor, data e motivo da alteração.
 - Relacione cada caso de teste a um requisito ou história de usuário.
 - Centralize a documentação em repositórios acessíveis a toda a equipe.
 - Versione os casos de teste como qualquer outro artefato de software (especialmente se integrados a sistemas de CI/CD).

Leitura Complementar

- **16 boas práticas em testes de software para acompanhar agora**

<https://esr.rnp.br/desenvolvimento-de-sistemas/melhores-praticas-em-testes-de-software/>

- **O que é teste unitário?**

<https://aws.amazon.com/pt/what-is/unit-testing/>

- **Diferentes tipos de testes de software**

<https://www.atlassian.com/br/continuous-delivery/software-testing/types-of-software-testing>